MASOODA · THE MATHS BEHIND THE SIMULATOR

From arrays to ADMM: every equation in MASooda, explained

This guide starts from what an array is and builds up to every equation the simulator uses: separation, consensus ADMM, resilient consensus, signal temporal logic, the battery circuit and the Kalman filter. Each idea comes with a picture, a worked example with real numbers from the code, the rule it obeys, and the exact function in the repository that computes it.

Numbers in the worked examples were computed with the repository's own functions on branch `bms/publishable` (configuration `configs/base_config.yaml`). Where the code and an equation differ in a detail, the code wins and the text says so.

0 How to read the notation

A few symbols appear everywhere. Learn these and the rest reads like sentences.

Symbol	Read it as	In MASooda
$x,\ t,\ d$	a plain number (scalar)	time t in s, a distance d in m
$\mathbf{p}_i$	a vector: a short list of numbers, here for drone i	position $\mathbf{p}_i = (x_i, y_i)$ in metres
$A,\ P,\ F$	a matrix: a table of numbers	adjacency matrix A, covariance P
$\lVert\mathbf{v}\rVert$	length (norm) of a vector	speed $\lVert\mathbf{v}\rVert$, distance $\lVert\mathbf{p}_i - \mathbf{p}_j\rVert$
$\mathbf{a}\cdot\mathbf{b}$ or $\mathbf{a}^\top\mathbf{b}$	dot product	how far $\mathbf{a}$ points along $\mathbf{b}$
$\sum_i,\ \min_i,\ \arg\min_L$	add over i; smallest value; the L that gives the smallest value	STL robustness, layer choice
$\mathcal{N}_i$	the set of neighbours of i	drones within 40 m over V2V
$\mathrm{clip}(x, a, b)$	x, but pushed back into $[a, b]$	`np.clip`
$\mathbf{G}\,\varphi,\ \mathbf{F}\,\varphi$	"always φ", "eventually φ"	STL safety and reachability
$\hat{x}$	an estimate of x	$\widehat{\mathrm{SoC}}$, the BMS estimate

Units are SI throughout: metres, seconds, metres per second, watts (W), watt-hours (Wh), volts (V), amperes (A), ohms (Ω), kelvin or °C for temperature differences.

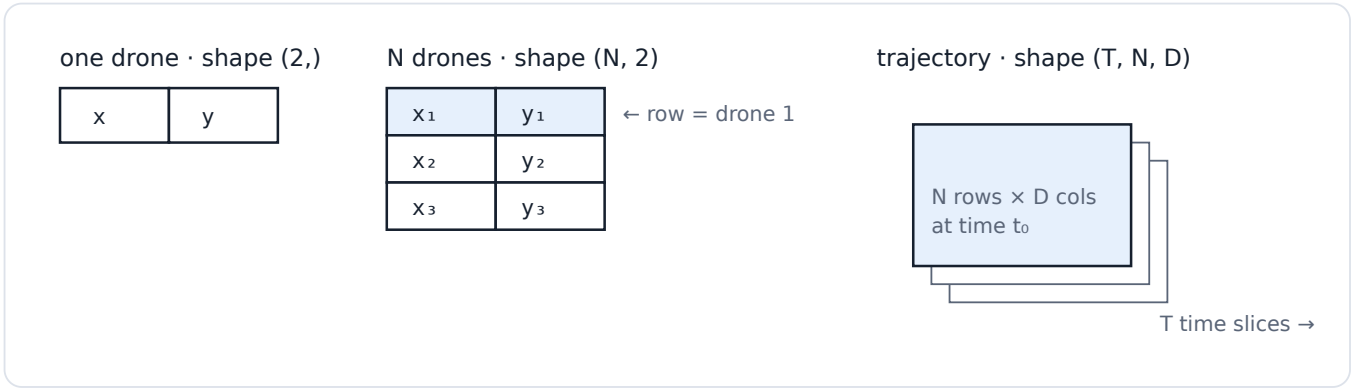
1 Arrays, vectors and matrices

The whole simulator is arithmetic on arrays. This chapter builds every tool the later chapters use.

1.1 Arrays and their shape

An **array** is a grid of numbers with a **shape**. NumPy, the library MASooda uses, writes the shape as a tuple:

- shape `(2,)`: one position (x, y);
- shape `(N, 2)`: the positions of N drones, one row each;
- shape `(T, N, D)`: a trajectory, T time samples × N drones × D coordinates. The STL monitor reads exactly this shape.



The same positions, seen at three scales. Most functions take `(N, 2)` or `(T, N, D)` arrays.

1.2 Vectors: adding, scaling, length

A vector is an arrow. Adding vectors puts arrows head to tail; multiplying by a number stretches the arrow.

$$\mathbf{a}+\mathbf{b}=(a_x+b_x,\ a_y+b_y), \qquad c\,\mathbf{a}=(c\,a_x,\ c\,a_y), \qquad \|\mathbf{a}\|=\sqrt{a_x^2+a_y^2}.$$

The length $\|\mathbf{a}\|$ is Pythagoras. The distance between two drones is the length of their difference:

$$d_{ij}=\|\mathbf{p}_i-\mathbf{p}_j\|.$$

A **unit vector** has length 1 and only carries a direction: $\hat{\mathbf{a}}=\mathbf{a}/\|\mathbf{a}\|$.

> **RULE · CLAMP A VECTOR**
> To keep a speed or acceleration under a limit m without changing its direction, rescale it when it is too long: $\mathbf{v}\leftarrow\mathbf{v}\cdot\min\big(1,\ m/\|\mathbf{v}\|\big)$.

In code `simulation/dynamics.py › clamp_vector_2d`

> **EXAMPLE**
> A command $\mathbf{v}=(12,\ 9)$ m/s has length $\sqrt{144+81}=15$ m/s. With the 15 m/s limit it passes unchanged. The command $(16,\ 12)$ has length 20 and is scaled by $15/20=0.75$ to $(12,\ 9)$.

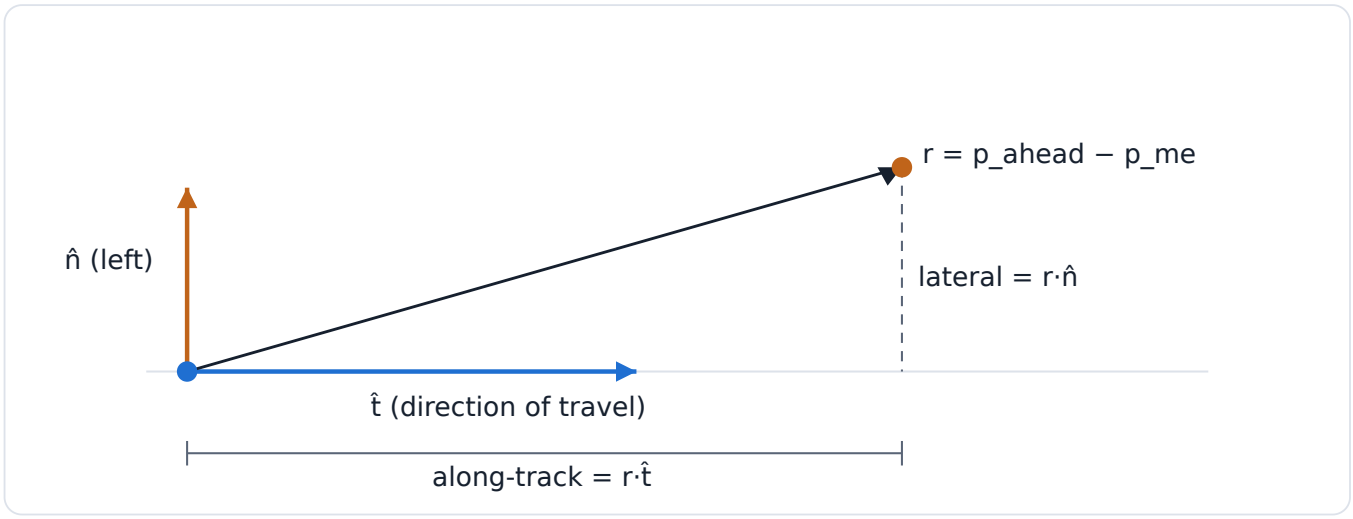
1.3 The dot product, projection and the "left" vector

The dot product multiplies matching entries and adds them:

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta.$$

If $\hat{\mathbf{t}}$ is a unit vector, $\mathbf{a} \cdot \hat{\mathbf{t}}$ is **how far a reaches along** $\hat{\mathbf{t}}$. MASooda uses this constantly: "along-track" distance to the drone ahead is $\mathbf{r} \cdot \hat{\mathbf{t}}$, and the sideways ("lateral") offset is $\mathbf{r} \cdot \hat{\mathbf{n}}$, where $\hat{\mathbf{n}}$ is the unit vector to the left of the direction of travel:

$$\hat{\mathbf{t}} = (t_x, t_y) \quad \Rightarrow \quad \hat{\mathbf{n}}_{\text{left}} = (-t_y,\ t_x).$$



Two dot products split the vector to another drone into "ahead of me" and "beside me".

> **EXAMPLE**
> I fly east, $\hat{\mathbf{t}} = (1, 0)$, so $\hat{\mathbf{n}} = (0, 1)$. A drone sits at $\mathbf{r} = (30, 4)$ m from me. Along-track $= 30 \cdot 1 + 4 \cdot 0 = 30$ m; lateral $= 30 \cdot 0 + 4 \cdot 1 = 4$ m. It is 30 m ahead, 4 m to my left, inside my 20 m tube (half-width 10 m), so it counts as traffic ahead.

In code `simulation/route_engine.py › _simulate (car-following: along = rel @ tan)`

`simulation/layer_agreement.py › LayerAgreement._gap`

1.4 Matrices

A matrix is a table of numbers that acts on vectors. Multiplying matrix M by vector $\mathbf{v}$ takes the dot product of each row with $\mathbf{v}$:

$$M\mathbf{v} = \begin{pmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} m_{11}v_1 + m_{12}v_2 \\ m_{21}v_1 + m_{22}v_2 \end{pmatrix}.$$

- The **transpose** $M^\top$ swaps rows and columns. A matrix with $M = M^\top$ is **symmetric**; adjacency matrices, Laplacians and covariance matrices all are.
- The **identity** I has ones on the diagonal and leaves vectors unchanged: $I\mathbf{v} = \mathbf{v}$.
- The **trace** $\operatorname{tr}(M)$ is the sum of the diagonal.
- An **eigenvalue** λ and eigenvector $\mathbf{u}$ satisfy $M\mathbf{u} = \lambda\mathbf{u}$: the matrix only stretches $\mathbf{u}$. A symmetric matrix has real eigenvalues; `np.linalg.eigvalsh` computes them. Chapter 4 uses the second-smallest eigenvalue of the graph Laplacian.

1.5 All pairwise distances at once: broadcasting

Many functions need every distance d_{ij} between N drones, an $N \times N$ matrix. NumPy builds it without loops by **broadcasting**: subtract a `(N,1,2)` copy from a `(1,N,2)` copy to get every difference in a `(N,N,2)` array, then take lengths along the last axis.

```
deltas    = positions[:, None, :] - positions[None, :, :]   # (N, N, 2)
distances = np.sqrt(np.sum(deltas * deltas, axis=2))        # (N, N)
```

> **EXAMPLE**
> Three drones at $(0,0)$, $(12,5)$, $(30,0)$:
>
> $$D = \begin{pmatrix} 0 & 13 & 30 \\ 13 & 0 & 18.68 \\ 30 & 18.68 & 0 \end{pmatrix} \text{ m}.$$
>
> $d_{12} = \sqrt{12^2+5^2} = 13$; $d_{23} = \sqrt{18^2+5^2} = 18.68$. With the 20 m ADMM detection radius, pairs (1,2) and (2,3) are in conflict and (1,3) is not. Section 4.4 uses this example again.

In code `conflict_resolution/detection.py › detect_conflicts`

`verification/stl_monitor.py › _pairwise_min`

`simulation/route_engine.py › layered_min_separation`

The route engine also uses `np.einsum("ijk,ijk->ij", diff, diff)`, which is the same sum of squares written as an index formula, and a uniform grid (`simulation/spatial_grid.py › UniformGrid`) so that it only compares drones in nearby cells. That turns $O(N^2)$ work into roughly $O(N)$ at 500 drones.

2 Motion of one drone

Every drone is a point that obeys a double integrator: acceleration changes velocity, velocity changes position.

2.1 The double integrator and Euler steps

Continuous motion is $\dot{\mathbf{p}} = \mathbf{v}$, $\dot{\mathbf{v}} = \mathbf{a}$. The simulator advances in small steps $\Delta t = 0.1$ s (forward Euler):

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{v}_t\,\Delta t, \qquad \mathbf{v}_{t+1} = \text{clamp}\big(\mathbf{v}_t + \text{clamp}(\mathbf{a}_t, a_{\max})\,\Delta t,\ v_{\max}\big),$$

with $v_{\max} = 15$ m/s. Altitude is not integrated: a drone sits on a discrete layer $L \in \{0,\dots,6\}$ whose height is $30 + 15L$ m.

2.2 Tracking a desired velocity

The higher layers (local rules, ADMM) produce a desired velocity $\mathbf{v}^{\text{des}}$. The engine turns it into an acceleration with a first-order lag of time constant $\tau_v = 0.5$ s:

$$\mathbf{a} = \frac{\mathbf{v}^{\text{des}} - \mathbf{v}}{\tau_v}.$$

The gap between actual and desired velocity shrinks by a factor $e^{-\Delta t/\tau_v}$ per step, roughly 18% per tick.

EXAMPLE

Flying $\mathbf{v} = (10, 0)$, desired $(12, 0)$: $\mathbf{a} = (2, 0)/0.5 = (4, 0)$ m/s². After one tick $\mathbf{v} = (10.4, 0)$ and the drone moved $10 \times 0.1 = 1$ m.

In code `simulation/dynamics.py › step_dynamics`

`simulation/route_engine.py › _simulate (accel = (desired − velocity) / tau_v)`

RULE · ORDER OF UPDATES

`step_dynamics` moves the position with the *old* velocity, then updates the velocity. The project audit (D21) noted that ADMM's one-step prediction $\mathbf{p} + \mathbf{v}^{\text{des}}\Delta t$ therefore differs slightly from the state the engine produces; the STL monitor measures the real outcome independently.

3 Corridors, maps and routes

Roads become a graph, routes become polylines, and the corridor tube is a distance rule around each polyline.

3.1 From map to local metres

Map coordinates are longitude λ and latitude φ in degrees. Over a few kilometres the Earth is flat enough for a local tangent plane around a reference point (λ_0, φ_0). A degree of latitude is about 111 km everywhere; a degree of longitude shrinks with $\cos\varphi_0$:

$$x = (\lambda - \lambda_0)\, m_{\text{lon}} \cos\varphi_0, \qquad y = (\varphi - \varphi_0)\, m_{\text{lat}}, \qquad m_{\text{lon}} \approx m_{\text{lat}} \approx 1.11 \times 10^5 \text{ m per degree.}$$

At Hyderabad ($\varphi_0 \approx 17.6°$), $\cos\varphi_0 \approx 0.953$, so a degree of longitude is about 106 km. The inverse, `to_geo`, divides by the same factors.

In code `simulation/road_network.py › GeoFrame.to_local / to_geo`

3.2 Shortest paths: Dijkstra

Junctions are nodes, road pieces are edges weighted by length. A route between two vertiports is the shortest path. Dijkstra's algorithm keeps a priority queue of tentative distances and repeatedly settles the closest unsettled node:

$$\text{dist}(v) \leftarrow \min\big(\text{dist}(v),\ \text{dist}(u) + w(u, v)\big) \quad \text{for each edge } (u, v).$$

On the campus graph (105 junctions) this takes well under a millisecond; on the Hyderabad graph (1,533 junctions) a few milliseconds.

In code `simulation/road_network.py › RoadNetwork.shortest_path`

`simulation/dtc_console.py › plan_route (avoids restricted areas)`

3.3 A route as an arc-length polyline

A route is a list of points $\mathbf{q}_0, \ldots, \mathbf{q}_K$. Each segment has length $\ell_k = \|\mathbf{q}_{k+1} - \mathbf{q}_k\|$ and unit direction $\hat{\mathbf{t}}_k$. The cumulative length $c_k = \sum_{m<k} \ell_m$ lets any distance s along the route map to a point:

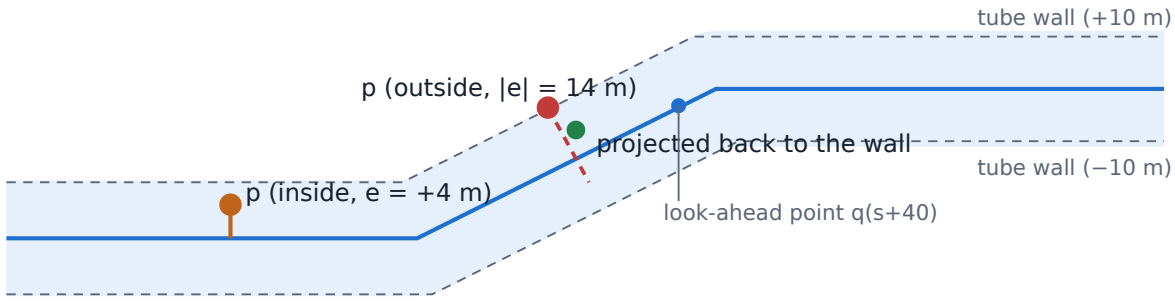
$$\mathbf{q}(s) = \mathbf{q}_k + \hat{\mathbf{t}}_k\,(s - c_k) \quad \text{for } c_k \le s < c_{k+1}.$$

3.4 Nearest point on a segment and the lateral error

To find where a drone at $\mathbf{p}$ is along its route, project onto each nearby segment, clip to the segment, and keep the closest:

$$\tau_k = \operatorname{clip}\big((\mathbf{p} - \mathbf{q}_k)\cdot\hat{\mathbf{t}}_k,\ 0,\ \ell_k\big), \qquad \mathbf{q}^* = \mathbf{q}_k + \tau_k\hat{\mathbf{t}}_k, \qquad e = (\mathbf{p} - \mathbf{q}^*)\cdot\hat{\mathbf{n}}.$$

e is the signed lateral error: positive to the left of the centreline. The search is limited to a window $[s - 40,\ s + 120]$ m around the last known progress so a route that crosses itself cannot make the drone "jump".



Each 20 m tube surrounds the route centreline. A drone more than 10 m off the line is put back on the wall and loses its outward velocity.

> **RULE · TUBE BOUNDARY**
> If $|e| > w/2$ (10 m), set $\mathbf{p} \leftarrow \mathbf{q}^* + \operatorname{sign}(e)\,\frac{w}{2}\,\hat{\mathbf{n}}$ and remove the outward velocity: $\mathbf{v} \leftarrow \mathbf{v} - \max(0, \mathbf{v}\cdot\hat{\mathbf{w}})\,\hat{\mathbf{w}}$, where $\hat{\mathbf{w}} = \operatorname{sign}(e)\hat{\mathbf{n}}$. The map edge is a geofence handled the same way.

In code `simulation/road_network.py › RoutePath.project` `RoutePath.point_at` `RoutePath.tangent_at` `RoadNetwork.clamp` `route_engine.py › _simulate (tube + geofence)`

3.5 Pure-pursuit steering with a keep-left lane

A drone steers towards a point 40 m ahead on its route, shifted left by a lane offset (a quarter of the tube width, 5 m; India keeps left). With reference speed v_{ref} and lateral gain $k_{\text{lat}} = 0.5\ \text{s}^{-1}$:

$$\mathbf{v}^{\text{goal}} = v_{\text{ref}}\,\frac{\mathbf{q}_{\text{look}} - \mathbf{p}}{\|\mathbf{q}_{\text{look}} - \mathbf{p}\|} \;-\; k_{\text{lat}}\,(e - e_{\text{lane}})\,\hat{\mathbf{n}}.$$

The second term is a proportional controller: 4 m off the lane line produces a 2 m/s correction towards it.

3.6 Sectors, polygons and restricted areas

The map's bounding box is cut into a 3 × 2 grid of sectors. With column $c = \lfloor 3(x - x_{\min})/(x_{\max} - x_{\min}) \rfloor$ and row $r = \lfloor 2(y - y_{\min})/(y_{\max} - y_{\min}) \rfloor$, the sector id is $3r + c \in \{0, \dots, 5\}$.

A restricted area (TFR) is a polygon. A point is inside if a ray from it crosses the boundary an odd number of times (the even–odd rule). A route segment enters a polygon if it crosses an edge, which is tested with the sign of the cross product $\text{orient}(\mathbf{a}, \mathbf{b}, \mathbf{c}) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x)$.

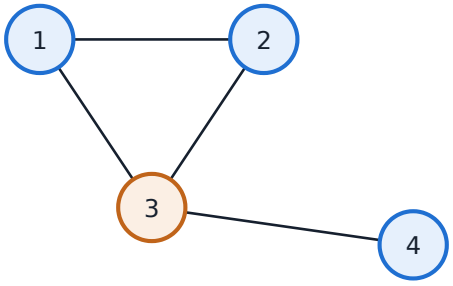
In code `route_engine.py › _sector` `dtc_console.py › points_in_polygon` `segments_cross_polygon` `route_hits_zones` `polygon_area (shoelace formula)`

4 Graphs and network theory

Who can talk to whom is a graph. Its matrix tells you how well the swarm can agree.

4.1 The communication graph

Each drone is a node. Two drones share an edge if they are within the V2V radius $R = 40$ m. The **adjacency matrix** A has $A_{ij} = 1$ for an edge, 0 otherwise; it is symmetric with zeros on the diagonal. The **degree** $\deg_i = \sum_j A_{ij}$ counts neighbours.



triangle 1–2–3 · drone 3 is the hub (degree 3)

A =

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

deg 2
deg 2
deg 3
deg 1

Row i of A lists drone i's neighbours. Summing a row gives its degree.

In code `simulation/comm_graph.py › build_radius_graph` `build_knn_graph` `spatial_grid.py › radius_adjacency`

4.2 The numbers in the Network panel

Measure	Formula	Example graph
Edges m	$\frac{1}{2}\sum_{ij} A_{ij}$	4
Average degree	$\frac{1}{n}\sum_i \deg_i = 2m/n$	2.0
Density	$\dfrac{2m}{n(n-1)}$: edges present out of all possible	0.667
Triangles	$\mathrm{tr}(A^3)/6$: a closed walk of length 3 is a triangle, counted 6 times	1
Connected triples	$\sum_i \binom{\deg_i}{2} = \sum_i \deg_i(\deg_i - 1)/2$	1+1+3+0 = 5
Transitivity	$3 \times$ triangles/triples: how often "my neighbours are neighbours"	0.6
Max degree centrality	$\max_i \deg_i /(n-1)$	1.0 (drone 3)
Components	groups reachable from each other (depth-first search)	1
Algebraic connectivity λ_2	second-smallest eigenvalue of $L = D - A$	1.0

In code `simulation/route_engine.py › _graph_metrics`

4.3 The Laplacian and why λ_2 matters

The graph Laplacian is $L = D - A$, where D is the diagonal matrix of degrees. For the example:

$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}, \qquad \text{eigenvalues } 0,\ 1,\ 3,\ 4.$$

- The smallest eigenvalue is always 0 (eigenvector: all ones), because each row of L sums to zero.
- The number of zero eigenvalues equals the number of connected components. So $\lambda_2 > 0$ exactly when the graph is connected; the code reports $\lambda_2 = 0$ for any disconnected graph.
- For averaging consensus $\dot{\mathbf{x}} = -L\mathbf{x}$, disagreement decays like $e^{-\lambda_2 t}$. A larger λ_2 means the swarm agrees faster. That is why it is called algebraic connectivity, or the Fiedler value.

EXAMPLE

Remove edge 3–4 and drone 4 is isolated: two components, two zero eigenvalues, $\lambda_2 = 0$. The Network panel would show "Components 2 (1 isolated)" and $\lambda_2 = 0$, while the giant-component value is still computed on drones 1–3.

4.4 The conflict graph and connected components

ADMM uses a second graph: an edge joins two drones on the **same layer** closer than 20 m. Each **connected component** of this graph is solved together. Breadth-first search finds components: start from a drone, visit its neighbours, then theirs, until nothing new appears.

In the three-drone example of 1.5, edges (1,2) and (2,3) put all three drones in one component even though drones 1 and 3 are 30 m apart. An earlier version split large components into chunks of five; that could separate two drones that were in conflict and silently drop their constraint (audit finding D05). The fix keeps every component whole.

In code `conflict_resolution/detection.py › detect_conflicts`

`conflict_resolution/adapter.py › resolve_all_conflicts`

`spatial_grid.py › close_same_layer_pairs`

5 Local rules: the decentralised tier

Each drone computes its own desired velocity from what it hears from neighbours.

5.1 Car-following: the time-headway law

A drone keeps a gap to the drone ahead in its lane that grows with speed: $g \geq d_0 + h\,v$. Solving for speed gives the rule

$$v_{\text{ref}} \leq \frac{g - d_0}{h}, \qquad d_0 = 12\ \text{m},\ h = 1\ \text{s}.$$

> **EXAMPLE**
> Gap 20 m: $v \leq (20 - 12)/1 = 8$ m/s, so a drone cruising at 12 m/s slows to 8. Gap 12 m or less: speed 0, it waits. Gap 30 m: limit 18 m/s, above cruise, so no effect.

The same law gives the layer-agreement "follow penalty" $\varphi = 1 - v_{\text{eff}}/v_c$, the fraction of cruise speed lost behind traffic on a layer (Chapter 8).

In code `route_engine.py › _simulate (v_ref = min(v_ref, (along - d_0) / h))`

`layer_agreement.py › follow_penalty` `route_formation.py › platoon_stats`

5.2 Boids: separation, alignment, cohesion

Reynolds' boids rules, as the code implements them:

$$\mathbf{s}_i = \sum_{j:\,0<d_{ij}<r_{\text{sep}}} \frac{\mathbf{p}_i - \mathbf{p}_j}{d_{ij}^2}, \qquad \mathbf{a}_i = \overline{\mathbf{v}}_{\mathcal{N}} - \mathbf{v}_i, \qquad \mathbf{c}_i = \left(\overline{\mathbf{p}}_{\mathcal{N}} - \mathbf{p}_i\right) \text{ along the road.}$$

- **Separation** pushes away from neighbours closer than $r_{\text{sep}} = 20$ m. Dividing the unit direction by distance once more ($1/d^2$ overall) makes close neighbours push much harder.
- **Alignment** steers towards the average velocity of same-direction neighbours.
- **Cohesion** pulls towards the average position, only along the road, and is switched off when any segment mate is closer than $\max(r_{\text{sep}},\ d_0 + hv)$, so it can never pull a drone inside its headway.

EXAMPLE

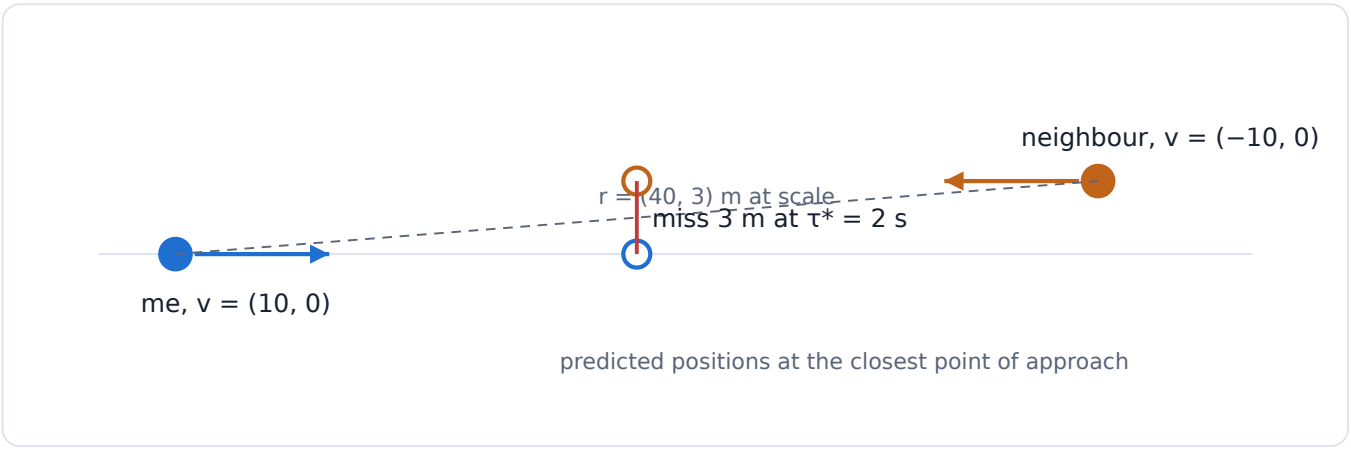
A neighbour 5 m to my right: $\mathbf{p}_i - \mathbf{p}_j = (0, 5)$, $d = 5$, so $\mathbf{s} = (0, 5)/25 = (0, 0.2)$. With weight $w_{\text{sep}} = 35$ this adds 7 m/s to the left. At 10 m the push is $35 \times 0.1 = 3.5$ m/s; at 20 m it stops.

In code `local_behavior/boids.py › separation` `alignment` `cohesion` `corridor_following` `compute_desired_velocity`

5.3 Closest point of approach (CPA)

For crossing or head-on traffic, a drone predicts where a neighbour will be if both keep their velocities. With relative position $\mathbf{r} = \mathbf{p}_j - \mathbf{p}_i$ and relative velocity $\mathbf{u} = \mathbf{v}_j - \mathbf{v}_i$, the distance at time τ is $\|\mathbf{r} + \mathbf{u}\tau\|$. Setting the derivative of its square to zero gives

$$\tau^* = -\frac{\mathbf{r} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}, \qquad \text{miss} = \|\mathbf{r} + \mathbf{u}\,\tau^*\|.$$



Projecting both drones forward finds when they will be closest and by how much.

EXAMPLE

$\mathbf{r} = (40, 3)$, my velocity $(10, 0)$, theirs $(-10, 0)$, so $\mathbf{u} = (-20, 0)$. $\tau^* = -(40 \cdot -20)/400 = 2$ s; miss $= \|(40, 3) + (-40, 0)\| = 3$ m. That is below $1.6\,d_{\text{min}} = 12.8$ m, so the drone adds an avoidance velocity away from the miss vector with urgency $2(12.8 - 3)/\max(\tau^*, 0.5) = 9.8$, capped at 6 m/s, and slows because the threat is ahead. Because 3 m < 8 m and $\tau^* < 3$ s, it is also flagged as a threat for a tactical altitude shift.

In code `route_engine.py › _simulate (tactical CPA avoidance, threat list)` `route_engine.py › _layer_transition_clear (10 s CPA check before a layer change)`

5.4 Putting the local velocity together

$$\mathbf{v}_i^{\text{des}} = \mathbf{v}_i^{\text{goal}} + \mathbf{v}_i^{\text{avoid}} + w_{\text{sep}}\,\mathbf{s}_i + 0.5\,w_{\text{ali}}\,\mathbf{a}_i + w_{\text{coh}}\,\mathbf{c}_i, \qquad w_{\text{sep}} = 35,\ w_{\text{ali}} = 0.4,\ w_{\text{coh}} = 0.1.$$

The goal term is already slowed by car-following and junction merging (a merge factor in $[0, 1]$ when a junction gap is blocked). ADMM may then overwrite $\mathbf{v}^{\text{des}}$ for drones in a conflict group.

In code `route_engine.py › _simulate` `route_formation.py › MergeTracker.factor`

6 Optimisation and consensus ADMM

When drones are about to get too close, they negotiate their next positions by solving a small optimisation problem together.

6.1 The problem each conflict group solves

Every drone i in a group has its current position $\mathbf{p}_i$ and an intended next position $\mathbf{t}_i = \mathbf{p}_i + \mathbf{v}_i \Delta t$. The group picks next positions $\mathbf{x}_i$ that stay as close as possible to the intentions while keeping apart:

$$\min_{\{\mathbf{x}_i\}} \sum_i \frac{w_i}{2} \|\mathbf{x}_i - \mathbf{t}_i\|^2 \quad \text{s.t.} \quad \hat{\mathbf{n}}_{ij} \cdot (\mathbf{x}_i - \mathbf{x}_j) \geq d_s \;\; \forall (i,j), \qquad \|\mathbf{x}_i - \mathbf{p}_i\| \leq v_{\max} \Delta t.$$

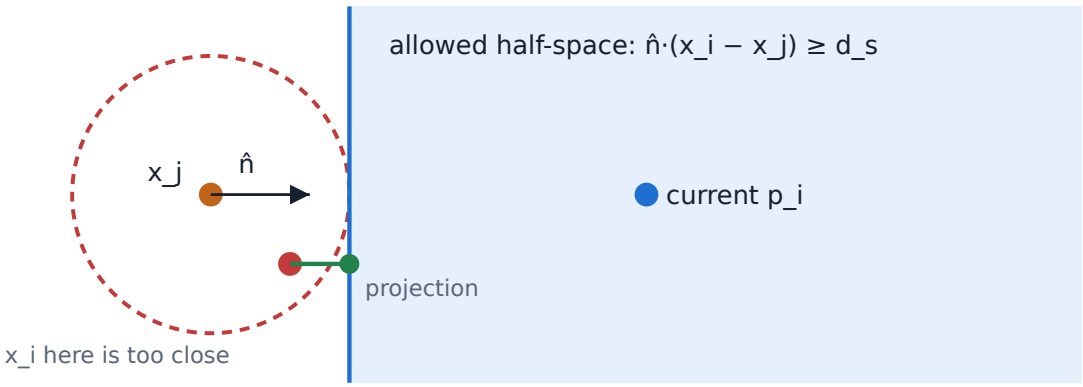
Here $d_s = 1.01\, d_{\min} = 8.08$ m and $v_{\max}\Delta t = 1.5$ m. The weights w_i say who should move less: 10 for emergency, 3 for eVTOL, 1 for delivery.

6.2 Convex sets and why we linearise

A set is **convex** if the straight line between any two of its points stays inside it. Minimising a convex bowl (sum of squares) over a convex set has one answer and fast algorithms. "Stay at least 8 m apart", $\|\mathbf{x}_i - \mathbf{x}_j\| \geq d_s$, is **not** convex: it is the outside of a circle. The code replaces it with a **half-space**, using the current direction between the drones $\hat{\mathbf{n}}_{ij} = (\mathbf{p}_i - \mathbf{p}_j)/\|\mathbf{p}_i - \mathbf{p}_j\|$:

$$\hat{\mathbf{n}}_{ij} \cdot (\mathbf{x}_i - \mathbf{x}_j) \geq d_s.$$

> **RULE · THE HALF-SPACE IS SAFE**
> For any unit vector $\hat{\mathbf{n}}$ and any vector $\mathbf{v}$, Cauchy–Schwarz gives $\hat{\mathbf{n}} \cdot \mathbf{v} \leq \|\mathbf{v}\|$. So if $\hat{\mathbf{n}} \cdot (\mathbf{x}_i - \mathbf{x}_j) \geq d_s$, then $\|\mathbf{x}_i - \mathbf{x}_j\| \geq d_s$. The linear rule is stricter than the real one, never looser.



dashed circle = true rule: stay outside 8.08 m of x_j (not convex)

The half-space sits entirely outside the circle, so satisfying it keeps the true separation.

Drones on different layers get no constraint at all (their layer index differs by at least 1), and a pair only gets one if it starts within $d_s + 2v_{\max}\Delta t = 11.08$ m: farther pairs cannot meet in one tick.

In code `conflict_resolution/constraints.py › build_pair_constraints` `PairConstraint`

6.3 Projection: the closest point in a set

ADMM only needs one operation per constraint: the Euclidean projection, the closest point inside the set.

- **Pair half-space.** If the pair violates the rule by $\delta = d_s - \hat{\mathbf{n}} \cdot (\mathbf{a} - \mathbf{b}) > 0$, move each drone half of it along $\pm\hat{\mathbf{n}}$:

$$\mathbf{a} \leftarrow \mathbf{a} + \tfrac{\delta}{2}\hat{\mathbf{n}}, \qquad \mathbf{b} \leftarrow \mathbf{b} - \tfrac{\delta}{2}\hat{\mathbf{n}}.$$

- **Speed ball.** If $\mathbf{x}$ is farther than $r = v_{\max}\Delta t$ from $\mathbf{p}$, pull it back to the ball: $\mathbf{x} \leftarrow \mathbf{p} + r\,(\mathbf{x} - \mathbf{p})/\|\mathbf{x} - \mathbf{p}\|$.

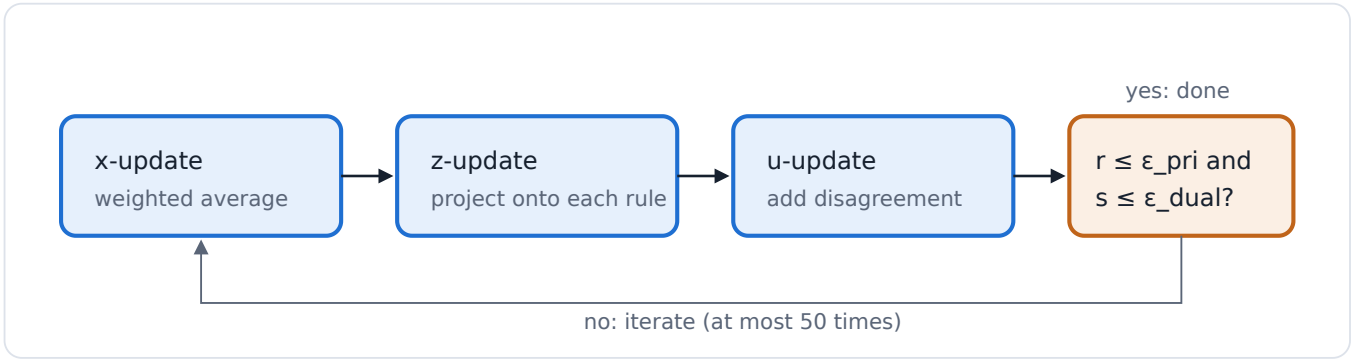
In code `constraints.py › project_pair` `project_ball`

6.4 Consensus ADMM, step by step

ADMM (alternating direction method of multipliers) splits a hard problem into easy pieces that agree with each other. Each drone i keeps its own decision $\mathbf{x}_i$. Every constraint k it takes part in keeps a **copy** $\mathbf{z}_{ik}$ of that decision that satisfies the constraint, plus a running disagreement (scaled dual variable) $\mathbf{u}_{ik}$. K_i is the number of copies drone i has: one speed ball plus one per pair constraint. Then repeat:

$$\begin{aligned}
&\textbf{x-update (each drone, closed form):} && \mathbf{x}_i = \frac{w_i\,\mathbf{t}_i + \rho \sum_k (\mathbf{z}_{ik} - \mathbf{u}_{ik})}{w_i + \rho K_i} \\
&\textbf{z-update (each constraint):} && \mathbf{z}_k = \mathrm{Proj}_{C_k}\big(\mathbf{x} + \mathbf{u}_k\big) \\
&\textbf{u-update (dual):} && \mathbf{u}_k \leftarrow \mathbf{u}_k + \mathbf{x} - \mathbf{z}_k
\end{aligned}$$

The x-update is a **weighted average** of the drone's own intention (weight w_i) and what its constraints want (weight ρ each). It comes from setting the gradient of $\frac{w_i}{2}\|\mathbf{x} - \mathbf{t}_i\|^2 + \frac{\rho}{2}\sum_k \|\mathbf{x} - \mathbf{z}_{ik} + \mathbf{u}_{ik}\|^2$ to zero. A heavy emergency drone ($w = 10$) stays near its intention and the others move.



One ADMM iteration. A conflict group typically converges in about 9–11 iterations.

6.5 When to stop: residuals

The **primal residual** $r = \big(\sum \|\mathbf{x} - \mathbf{z}\|^2\big)^{1/2}$ measures how far decisions are from their rule-satisfying copies. The **dual residual** $s = \rho\big(\sum \|\mathbf{z}^{\text{new}} - \mathbf{z}^{\text{old}}\|^2\big)^{1/2}$ measures whether the copies are still moving. Stop when both are small:

$$r \le \sqrt{m}\,\varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \max(\|\mathbf{x}\|, \|\mathbf{z}\|), \qquad s \le \sqrt{m}\,\varepsilon_{\text{abs}} + \varepsilon_{\text{rel}}\,\rho\,\|\mathbf{u}\|,$$

with $m = \sum_i K_i$, $\varepsilon_{\text{abs}} = 10^{-3}$, $\varepsilon_{\text{rel}} = 10^{-4}$, $\rho = 1$, at most 50 iterations. If it has not converged after 50, the group keeps its original intended velocities and counts as a **fallback**. An unverified iterate is never

used (audit finding D07).

In code `conflict_resolution/solver.py › ADMMSolver.solve` `ADMMSolver.from_config` `adapter.py › resolve_conflict_event` `simulation/admm_probe.py › ADMMProbe (live timer)`

6.6 Worked example: two drones head-on

EXAMPLE · COMPUTED WITH ADMMSOLVER

Drone 1 at $x = 0$ flies east at 12 m/s; drone 2 at $x = 9$ flies west at 12 m/s, same layer. Intentions after one tick: $t_1 = 1.2$, $t_2 = 7.8$, a gap of only 6.6 m, but $d_s = 8.08$ m. The constraint normal is $\hat{\mathbf{n}} = (-1, 0)$, so the rule reads $x_2 - x_1 \geq 8.08$.

Iteration 1 by hand. Copies start at the current positions, duals at zero, $K_1 = K_2 = 2$ (ball + pair):

$$x_1 = \frac{1 \cdot 1.2 + 1 \cdot (0 + 0)}{1 + 2} = 0.40, \qquad x_2 = \frac{7.8 + (9 + 9)}{3} = 8.60.$$

The gap $8.2 \geq 8.08$ already holds, so the pair copy accepts $(0.40,\ 8.60)$ unchanged; both are inside their 1.5 m speed balls. The primal residual is 0, but the copies moved from $(0, 9)$ to $(0.4, 8.6)$, so $s = \sqrt{0.4^2 \cdot 2 + 0.4^2 \cdot 2} = 0.8$: not converged yet.

Result. Equal weights: converged in 13 iterations to $x_1 = 0.460$, $x_2 = 8.540$, gap 8.080 m. Each drone gave up exactly half of the 1.48 m shortfall (0.74 m each), which is the exact solution of the problem.
Emergency drone 1 ($w = 10$) vs. delivery drone 2 ($w = 1$): converged in 18 iterations to $x_1 = 1.066$, $x_2 = 9.146$. The shortfall is shared in proportion $1/w$: drone 1 gives up 0.134 m, drone 2 gives up 1.346 m, ten times more.

Iteration	primal r (equal)	dual s (equal)	primal r (10 : 1)	dual s (10 : 1)
1	0.000	0.800	0.339	1.328
2	0.292	0.387	0.504	0.166
3	0.349	0.057	0.430	0.101
4	0.251	0.097	0.313	0.166
5	0.135	0.116	0.203	0.167
6	0.051	0.084	0.115	0.138

Try it: the same solver, running in your browser

This is a line-by-line port of `ADMMSolver.solve` for two drones flying at each other on one layer. Change the starting gap or the weights and step through the iterations.

Starting gap (m)

9.0

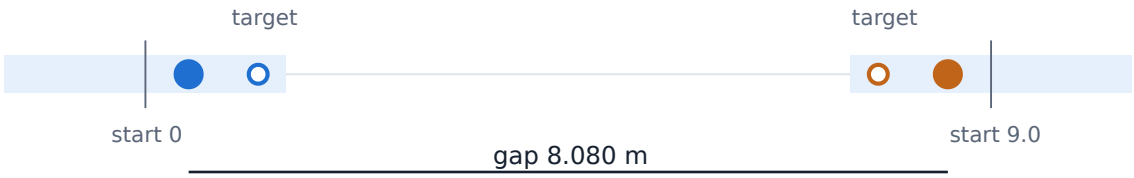
Weights (drone 1 : drone 2)

1 : 1 (delivery vs delivery)

Iteration

13

shaded: each drone's 1.5 m speed ball · hollow: intended position · filled: ADMM iterate



iteration 13/13 (converged) · x_1 = 0.460 m, x_2 = 8.540 m · drone 1 gave up 0.740 m, drone 2 gave up 0.740 m · primal r = 0.0002, dual s = 0.0017 · weights 1 : 1

6.7 Health-aware weights

The class weight can rise with a drone's energy shortage. With energy margin $m = E^{\mathrm{BMS}}/E^{\mathrm{need}}$ (Chapter 13), reference margin $m_{\mathrm{ref}} = 2$ and top weight $w^{\mathrm{top}} = 10$:

$$u = \mathrm{clip}\Big(\frac{m_{\mathrm{ref}} - m}{m_{\mathrm{ref}} - 1},\, 0,\, 1\Big), \qquad w = w^c + (w^{\mathrm{top}} - w^c)\,u,$$

with $u = 1$ for a drone at BMS CRITICAL or FAULT.

EXAMPLE

A delivery drone ($w^c = 1$) with margin $m = 1.5$: $u = (2 - 1.5)/1 = 0.5$, so $w = 1 + 9 \times 0.5 = 5.5$. With $m \geq 2$ it keeps $w = 1$; with $m \leq 1$ it gets 10, the same as an emergency.

RULE · WHAT THE WEIGHTS CAN AND CANNOT DO

(i) $w^c \leq w \leq w^{\mathrm{top}}$; (ii) w never increases as the margin grows; (iii) the feasible set does not depend on w, so weights only choose which feasible answer is picked, never whether separation holds; (iv) the share a drone keeps of its own target in the x-update, $w/(w + \rho K)$, rises with w.

In code `route_engine.py › _health_tick (health_w)`

`adapter.py › resolve_conflict_event (weights)`

`configs/base_config.yaml › admm.priority_weights, admm.health_weight`

7 Consensus, attacks and W-MSR

Averaging is how a swarm agrees. One liar can break plain averaging; W-MSR stops that.

7.1 Averaging consensus as a matrix

Each honest drone replaces its value by the average of its own and its neighbours' values:

$$x_i(k+1) = \frac{1}{1+\deg_i}\Big(x_i(k) + \sum_{j\in\mathcal{N}_i} x_j(k)\Big) \quad \Longleftrightarrow \quad \mathbf{x}(k+1) = W\mathbf{x}(k),$$

where $W = (I+D)^{-1}(I+A)$. Every row of W is non-negative and sums to 1 (it is **row-stochastic**), so each new value lies between the smallest and largest old values. On a connected graph all values converge to one common value.

7.2 What a liar does

A malicious drone that always reports a fixed value c never updates. Its neighbours keep averaging c in, and on a connected graph every honest value is dragged to c. In the position-spoofing attack, the liar broadcasts its true position plus an offset (constant, 25–30 m in the presets, or growing over time at a set rate).

In code `resilience/consensus.py › plain_average` `run_consensus` `resilience/attack.py › inject_attack` `apply_attack`

7.3 The W-MSR rule

W-MSR (weighted mean-subsequence-reduced) assumes at most f liars in any neighbourhood. Before averaging, drone i sorts the reports and removes up to f values strictly **above** its own value (the largest ones) and up to f strictly **below** (the smallest ones). It averages its own value with what is left.

$$x_i(k+1) = \text{mean}\Big(\{x_i(k)\} \cup \mathcal{R}_i(k)\Big), \quad \mathcal{R}_i = \text{reports minus the } f \text{ largest above } x_i \text{ and the } f \text{ smallest below } x_i.$$

If there are $2f$ or fewer reports, the code takes the median of own value and reports instead.



With $f = 1$: the result is mean(10, 11, 12) = 11. Plain averaging would give 18.4.

EXAMPLE · COMPUTED WITH WMSR_SCALAR

Own value 10, reports 9, 11, 12, 50, $f = 1$. Above 10: $\{11, 12, 50\}$, drop the largest one, keep 11 and 12. Below 10: $\{9\}$, drop it. Result $\text{mean}(10, 11, 12) = 11.0$. Plain averaging: $(10 + 9 + 11 + 12 + 50)/5 = 18.4$. One liar moved the plain average by 8.4; it barely moved W-MSR.

RULE · WHEN W-MSR IS GUARANTEED TO WORK

The classic result (LeBlanc et al., 2013) needs the graph to be $(2f+1)$-robust: roughly, every group of drones has some member with at least $2f+1$ neighbours outside the group. With sparse V2V graphs (the live Network panel often shows average degree near 1) that condition does not hold everywhere, so MASooda reports W-MSR's effect empirically, not as a guarantee.

In code `resilience/wmsr.py › wmsr_scalar` `wmsr_vector (per coordinate)` `consensus.py › run_wmsr_consensus` `layer_agreement.py › LayerAgreement.round (W-MSR on layer prices)`

Try it: W-MSR on your own numbers

Own value: 10

Neighbour reports (comma separated): 9, 11, 12, 50

f (liars tolerated): 1

own 10 | ~~9~~ | 11 | 12 | ~~50~~

W-MSR (f = 1) = 11.000 · plain average = 18.400

7.4 The trusted-range check

Before W-MSR, each receiver compares the distance a report claims with the distance its own sensor measures. If they disagree by more than 3 m, the report is moved along its bearing to the sensed range. The neighbour is **not** deleted, because deleting it would make the receiver blind to a real aircraft:

$$\left|\,\|\mathbf{p}_j^{\text{rep}}-\mathbf{p}_i\|-\rho_{ij}\right|>3\text{ m}\;\Rightarrow\;\mathbf{p}_j^{\text{corr}}=\mathbf{p}_i+\rho_{ij}\,\frac{\mathbf{p}_j^{\text{rep}}-\mathbf{p}_i}{\|\mathbf{p}_j^{\text{rep}}-\mathbf{p}_i\|}.$$

EXAMPLE

I am at the origin. A neighbour reports $(40, 30)$, 50 m away, but my range sensor reads $\rho = 20$ m. The difference is 30 m, more than 3 m, so the corrected position is $20 \times (0.8, 0.6) = (16, 12)$. Separation then reacts to a drone 20 m away, which is the truth along that bearing.

In code `resilience/defense.py › defend_neighbor_reports` `resilience/spoofer_estimate.py › estimate_spoofer_position`

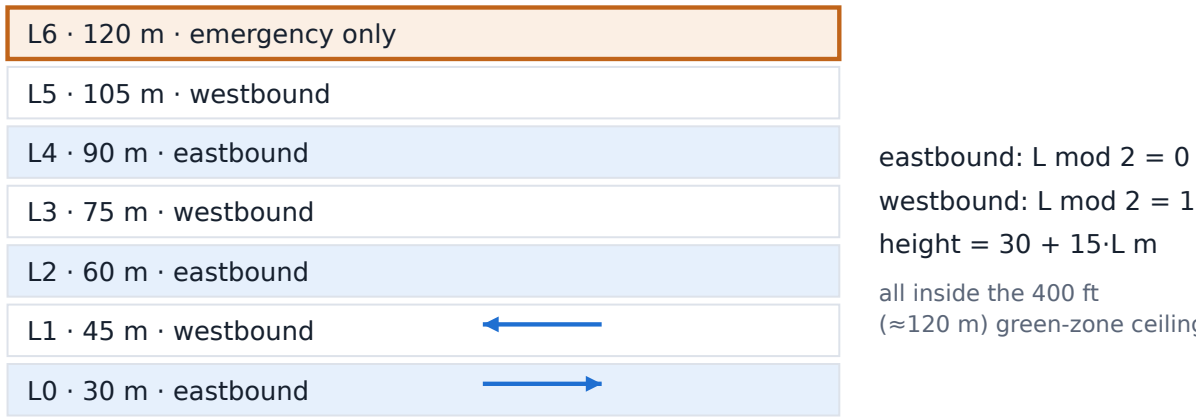
The audit (D02) records that the sensed range ρ_{ij} comes from the simulator's true positions, i.e. an ideal range sensor. That is a modelling assumption, stated as such.

8 The global tier: the DTC and altitude layers

The Drone Traffic Controller counts traffic in a sector × layer grid and moves drones out of crowded cells, slowly and only when it is safe.

8.1 The occupancy matrix

Every 2 s the DTC fills a 6×7 matrix N, where $N_{s,L}$ is the number of airborne cooperative drones in sector s on layer L. The capacity per cell is $C = 15$. This is the "sector × layer load" table in the DTC console.



The semicircular rule splits directions by layer parity, so head-on traffic is always one layer apart.

8.2 Priority-aware allocation

The DTC processes drones in priority order (emergency first, then eVTOL, then delivery; ties by id). An emergency goes to the reserved layer. Every other drone may only use layers allowed for its direction, $\mathcal{A}_d = \{L \neq L_{\text{emg}} : L \bmod 2 = d\}$, and keeps its current cell while that cell has room; otherwise it is offered the emptiest allowed cell. Every proposed move passes a **safety gate**: the target layer must have nobody within $3d_{\min}$ now and no CPA closer than $2d_{\min}$ within 10 s. Otherwise the move is deferred (the "gated" count).

In code `simulation/overseer.py › GlobalOverseer.update (priority_aware)`

`overseer_link.py › OverseerLink` `route_engine.py › _layer_transition_clear`

8.3 Layer agreement: choosing a layer by cost

Each drone also negotiates its layer with neighbours. For each allowed layer L it scores

$$J_i(L) = w_f\,\varphi_i(L) + w_p\,\lambda_i(L) + w_e\,c_i(L) + w_s[L \neq L_i] + w_{\text{dir}}[L \notin \mathcal{A}_d],$$

- $\varphi_i(L) = 1 - v_{\text{eff}}/v_c$ with $v_{\text{eff}} = \min\big(v_c, \max(0, (g_i(L) - d_0)/h)\big)$: speed lost behind the drone ahead on L (1 if traffic is head-on);
- $\lambda_i(L)$: the congestion price of (my sector, L);
- $c_i(L) = \max(0, L - L_i)\,H/H_{\text{ref}}\,\big(1 + k_b(1 - \text{SoC}_i)\big)$: climb energy, costlier on a low battery;
- $[\cdot]$ is 1 when the condition is true: a small cost for switching, and a penalty for the wrong-parity layer.

The drone wants $L_i^* = \arg\min_L J_i(L)$. It moves only if the gain $J_i(L_i) - J_i(L_i^*)$ exceeds a threshold, no neighbour with a stronger claim wants the same layer (larger gain, then higher priority, then lower id), and the safety gate passes. So at most one drone enters a layer per neighbourhood per round.

8.4 Agreeing on prices: consensus-based dual ascent

Prices are the dual variables of the constraint "load ≤ target". Each round (1 s), every drone averages its neighbours' price vectors (with W-MSR when the defence is on) and nudges each entry by how overloaded it hears that layer is:

$$\lambda_i(L) \leftarrow \mathrm{clip}\Big(\,\mathrm{mean}_{j\in\mathcal{N}_i\cup\{i\}}\,\lambda_j(L) + \alpha\,\frac{\ell_i(L)-\kappa}{\kappa},\;0,\;\lambda_{\max}\Big).$$

When the DTC link is up, the DTC overrides the estimate with the true global count:

$$\lambda^{\mathrm{DTC}}_{s,L} = \frac{\max\big(0,\;N_{s,L}/C - \theta\big)}{1-\theta}.$$

Configured values: $\alpha = 0.3$, target load $\kappa = 2$ drones per layer within V2V range, $\lambda_{\max} = 2$, $\theta = 0.6$.

> **EXAMPLE**
> With $C = 15$ and $\theta = 0.6$: a cell with 6 drones has load 0.4 and price 0; 12 drones give $(0.8 - 0.6)/0.4 = 0.5$; 15 drones give 1.0. Prices stay at zero until a cell is 60% full, then climb linearly. On the V2V side, a drone that hears 4 drones on a layer ($\ell = 4$, $\kappa = 2$) with neighbour prices averaging 0.2 updates to $0.2 + 0.3 \times (4-2)/2 = 0.5$. With the link down, drones keep agreeing among themselves, so congestion pricing degrades gracefully.

In code `simulation/layer_agreement.py › LayerAgreement.round` `dtc_seed_prices` `dtc_resolve` `direction_layers` `follow_penalty` `climb_cost` `departure_layer`

9 Signal temporal logic: measuring safety

STL turns "the drones are always at least 8 m apart" into a number that says how safely the rule holds.

9.1 Signals, predicates and operators

A **signal** is a value over time, here $d_{ij}(t)$. A **predicate** is a test on it, $d_{ij}(t) \geq d_{\min}$. STL adds time operators:

- $\mathbf{G}\,\varphi$ ("globally" or "always"): φ holds at every time in the window;
- $\mathbf{F}\,\varphi$ ("finally" or "eventually"): φ holds at some time in the window.

MASooda's safety specification is $\varphi_{\mathrm{safe}} = \mathbf{G}\,\big(d_{ij}(t) \geq d_{\min}\big)$ for all pairs $i \neq j$, with $d_{\min} = 8$ m.

9.2 Robustness: how much margin is left

Quantitative semantics give each formula a signed number ρ: positive means satisfied with that much margin, negative means violated by that much. The rules are mechanical: a predicate $f \geq c$ has robustness $f - c$; "and" and "always" take the minimum; "or" and "eventually" take the maximum.

$$\rho(t) = \min_{i\neq j}\big(d_{ij}(t) - d_{\min}\big), \qquad \rho\big(\mathbf{G}\varphi\big) = \min_t \rho(t), \qquad \rho\big(\mathbf{F}\varphi\big) = \max_t \rho(t).$$



The Safety panel plots exactly this $\rho(t)$. A dip below zero is a loss of separation; its depth says by how much.

EXAMPLE

Three snapshots of the closest pair: 16.1 m, 9.0 m, 6.5 m. Then $\rho = 8.1,\ 1.0,\ -1.5$ m and $\rho(\mathbf{G}\varphi) = -1.5$ m: violated, by 1.5 m, at the third sample.

9.3 Layer-aware distance

Two drones on different layers are not close even if they overlap on the map. The final system measures a 3D distance with the layer gap converted to metres ($H = 15$ m per layer):

$$d_{ij} = \big\|(\Delta x,\ \Delta y,\ \Delta L \cdot H)\big\|.$$

EXAMPLE

Drones directly above each other on L0 and L1: $d = \sqrt{0 + 0 + 15^2} = 15$ m, $\rho = +7$ m. On the same layer 5 m apart: $\rho = -3$ m. The earlier XY-only metric (audit D01) would have scored both pairs as violations; that is why the Week 6 violation counts were inflated.

9.4 Reachability: "eventually arrive"

$$\rho\big(\mathbf{F}(\|\mathbf{p}_i - \mathbf{g}_i\| \le r)\big) = \min_i \max_t \big(r - \|\mathbf{p}_i(t) - \mathbf{g}_i\|\big).$$

"Every drone eventually gets within r of its goal": max over time (eventually), min over drones (for all).

In code `verification/stl_monitor.py › instantaneous_robustness` `robustness_series` `stl_monitor` `SlidingSTLMonitor` `liveness_robustness` `verification/metrics.py › robustness_series (layer_height_m)` `route_engine.py › layered_min_separation` `live_monitor.py › LiveSTLMonitor`

A **near miss** is one loss-of-separation episode: a run of consecutive ticks with $\rho < 0$. A **crash** is a pair closer than 2 m. Recovery time is how long an episode lasts until $\rho \ge 0$ holds for at least 1 s (`metrics.py › recovery_times`).

10 Flight power from momentum theory

How many watts a drone needs depends on its weight, rotor area, speed and the air density.

10.1 Air density

The standard atmosphere gives the pressure at elevation h, and the ideal gas law gives the density:

$$p = 101325\,(1 - 2.25577 \times 10^{-5} h)^{5.25588}\ \text{Pa}, \qquad \rho_a = \frac{p}{R_{\text{air}}\,T}, \quad R_{\text{air}} = 287.05\ \tfrac{\text{J}}{\text{kg K}}.$$

> **EXAMPLE · KANDI, 542 M**
> $\rho_a = 1.148$ kg/m³ at 15 °C, 1.074 at 35 °C and 1.043 at 44 °C. Hot, thin air means more power to hover: at 35 °C hover power is 3.4% higher than at 15 °C, which is the number the Setup panel shows.

In code `simulation/drone_health.py › air_density`

10.2 Hover, induced velocity and forward flight

Rotors push air down. Momentum theory says that to make thrust $T = mg$ over disc area A, the air through the disc must move at the hover-induced velocity

$$v_h = \sqrt{\frac{T}{2\rho_a A}}.$$

In forward flight at airspeed v, the rotor meets fresh air and needs less induced velocity:

$$v_i = \sqrt{-\tfrac{v^2}{2} + \sqrt{\tfrac{v^4}{4} + v_h^4}}.$$

Mechanical power is induced power (divided by the figure of merit FM) plus drag on the body; electrical power divides by propulsion efficiency η and motor health h_m and adds avionics:

$$P_{\text{mech}} = \frac{T\,v_i}{\text{FM}} + \tfrac{1}{2}\rho_a C_D A_f\,v^3\ (+\,T\,v_{\text{climb}}\ \text{while climbing}), \qquad P_{\text{el}} = \frac{P_{\text{mech}}}{\eta\,h_m} + P_{\text{av}}.$$

> **EXAMPLE · DELIVERY DRONE AT 35 °C**
> $m = 8$ kg, so $T = 78.45$ N. Four 0.38 m rotors: $A = 4\pi(0.19)^2 = 0.454$ m². $v_h = \sqrt{78.45/(2 \cdot 1.074 \cdot 0.454)} = 8.97$ m/s.
> Hover ($v = 0$): $P_{\text{mech}} = 78.45 \times 8.97/0.65 = 1083$ W, so $P_{\text{el}} = 1083/0.8 + 25 = 1379$ W.
> Cruise at 10.8 m/s: $v_i = 6.41$ m/s, $P_{\text{mech}} = 773.8 + 54.1 = 828$ W, $P_{\text{el}} = 1060$ W. Cruising costs less than hovering, because forward flight feeds the rotor fresh air.

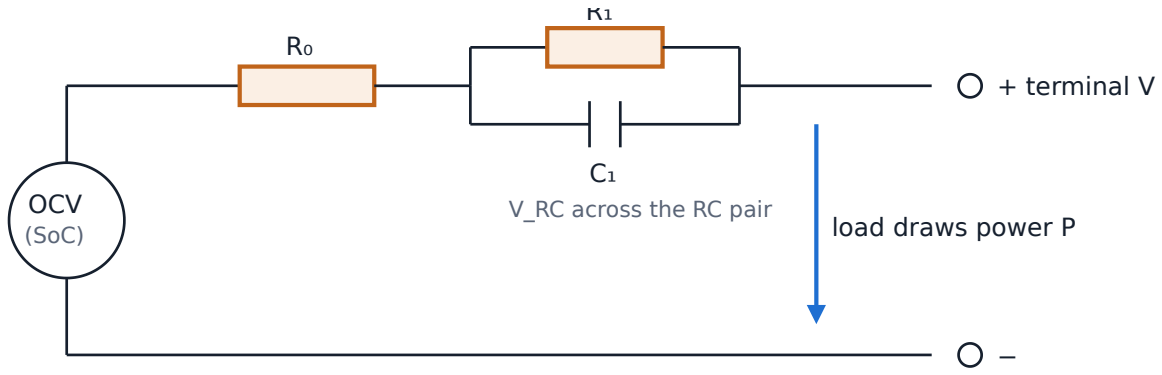
Since $P_{\text{el}}(v)$ falls and then rises with speed (a "power bucket"), there is a minimum-power speed. A drone whose weakest cell is near cut-off flies at that speed (`min_power_speed`).

In code `drone_health.py › HealthModel.power` `HealthModel._hover_mech` `min_power_speed` `climb_power_w`

11 The battery: a 1-RC equivalent circuit

A lithium pack behaves like a voltage source with resistors and one capacitor. That small circuit predicts voltage sag, heat and remaining energy.

11.1 The circuit



Open-circuit voltage depends on state of charge. R_0 causes instant sag; the R_1C_1 pair causes slow polarisation with time constant $\tau = R_1C_1 = 20$ s.

The terminal voltage is $V = \mathrm{OCV} - V_{RC} - IR_0$. The drone asks for an electrical power P, so $P = VI = (E - IR_0)I$ with $E = \mathrm{OCV} - V_{RC}$. That is a quadratic in I:

$$R_0 I^2 - EI + P = 0 \quad \Rightarrow \quad I = \frac{E - \sqrt{E^2 - 4R_0P}}{2R_0}.$$

The smaller root is the physical one. If $E^2 < 4R_0P$ there is no real root: the pack cannot deliver that power (the BMS flags it as a current FAULT).

> **EXAMPLE**
> $E = 23.5$ V, $R_0 = 0.02\ \Omega$, $P = 1219$ W: $I = (23.5 - \sqrt{552.25 - 97.52})/0.04 = 54.4$ A and $V = 23.5 - 54.4 \times 0.02 = 22.41$ V. The live site showed 22.3 V · 54.8 A · 1219 W for drone D-37.

11.2 Updating the state over one step

The RC voltage relaxes exponentially towards $R_1 I$; state of charge falls by the charge drawn (coulomb counting):

$$V_{RC}^{+} = V_{RC}\, e^{-\Delta t/\tau} + R_1\left(1 - e^{-\Delta t/\tau}\right) I, \qquad \mathrm{SoC}^{+} = \mathrm{SoC} - \frac{(I + I_f)\,\Delta t}{3600\,Q},$$

with $R_1 = 0.5R_0$, Q the capacity in Ah (scaled by state of health), and I_f an internal fault current that drains charge without reaching the current sensor (the overheating fault). Resistance rises as the pack ages and in the cold: $R_0 = R_{\mathrm{pack}}\,(1 + 0.5(1 - \mathrm{SoH}))\,(1 + \alpha_R \max(0, 25 - T_p))$.

In code `drone_health.py › HealthModel._discharge` `HealthModel._r` `HealthModel._update_ocv`

11.3 Open-circuit voltage by interpolation

Each cell's OCV is a table of (SoC, volts) pairs joined by straight lines, which is piecewise-linear interpolation:

SoC %	0	5	10	20	40	60	80	100
OCV (V/cell)	3.00	3.30	3.45	3.60	3.72	3.85	4.00	4.20

Between table points $\mathrm{OCV}(s) = V_k + (V_{k+1} - V_k)\frac{s - s_k}{s_{k+1} - s_k}$. At 50%: $3.72 + 0.13 \times \frac{10}{20} = 3.785$ V per cell. The pack OCV is the sum of the n_s cells in series.

In code `drone_health.py › HealthModel._ocv_cell (np.interp)` `_pack_ocv`

`configs/base_config.yaml › ocv_cell_v`

11.4 Weakest-cell usable energy

Cells in series carry the same current, but each has a slightly different charge offset δ_k. The pack must stop when its **weakest** cell reaches $\mathrm{SoC}_{\min} = 5\%$. Every cell gives up the same span $\Delta = \min_k s_k - \mathrm{SoC}_{\min}$, where $s_k = \mathrm{SoC} + \delta_k$, and the energy is the area under each cell's OCV curve over that span:

$$E^{\mathrm{use}} = Q \sum_{k=1}^{n_s} \int_{s_k - \Delta}^{s_k} \mathrm{OCV}(s)\, ds.$$

The integral of a piecewise-linear function is computed once as a running table $F(s) = \int_0^s \mathrm{OCV}$, so each call is $Q \sum_k \big(F(s_k) - F(s_k - \Delta)\big)$. One bad cell (a cell-failure fault: −15% charge, 4× resistance) shrinks the span for the whole pack.

In code `drone_health.py › HealthModel._usable_wh` `e_rem_wh (true)`

`e_rem_bms_wh (estimated, Chapter 13)`

11.5 Temperature: first-order heat balance

Heat in minus heat out, divided by heat capacity:

$$C_p \dot{T}_p = I^2 R_0 + I V_{RC} + q_f - \big(h_0 + h_1\sqrt{v}\big)(T_p - T_{\mathrm{amb}}), \qquad C_m \dot{T}_m = (1 - \eta h_m) P_{\mathrm{el}}^{\mathrm{prop}} - \big(h_{m0} + h_{m1}\sqrt{v}\big)(T_m - T_{\mathrm{amb}}).$$

Airflow ($\sqrt{v}$) cools: a hovering drone runs hotter than a cruising one. Steady state is where heat in equals heat out; for the delivery drone at cruise the pack sits about 7 K above ambient.

11.6 BMS states with hysteresis

The BMS gives each category a level 0–3 (NORMAL, WARNING, CRITICAL, FAULT) by comparing with thresholds, e.g. pack temperature > 55 / 65 / 75 °C, SoC < 30 / 15 %, any cell < 3.0 V, imbalance > 50 / 100 mV. The drone's status is the worst category. To stop flickering, a level only clears once the value is a margin past the threshold (2% SoC, 3 °C, 10 mV): **hysteresis**.

In code `drone_health.py › HealthModel._level` `HealthModel.bms`

`configs/base_config.yaml › health.thresholds`

12 Estimation: the extended Kalman filter

The BMS cannot see the true charge. It infers it from noisy voltage and current, and it knows how unsure it is.

12.1 Gaussian uncertainty

An estimate is a mean $\hat{x}$ plus a variance σ^2 (or, for several numbers, a covariance matrix P). For a Gaussian error, the true value lies below $\hat{x} - z\sigma$ with probability $1 - \Phi(z)$, where Φ is the standard normal cumulative distribution. For $z = 1.645$, that probability is 5%.

12.2 The Kalman filter in one dimension

Each step has two halves. **Predict** with the model, which adds uncertainty q. **Update** with a measurement of variance r, weighting it by the Kalman gain k:

$$\text{predict: } \hat{x}^- = f(\hat{x}),\ \ p^- = p + q; \qquad \text{update: } k = \frac{p^-}{p^- + r},\ \ \hat{x} = \hat{x}^- + k\,(y - \hat{y}),\ \ p = (1-k)\,p^-.$$

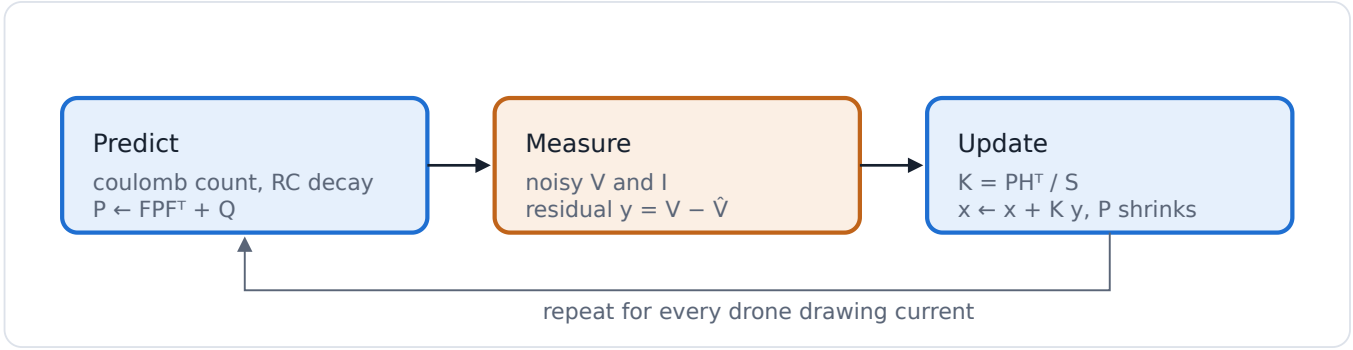
> **EXAMPLE**
> Prior SoC estimate 50% with $p^- = 4$ (σ = 2 points). A voltage reading implies 47% with $r = 1$. Then $k = 4/5 = 0.8$, the new estimate is $50 + 0.8(47 - 50) = 47.6\%$, and $p = 0.2 \times 4 = 0.8$ (σ ≈ 0.9). The filter trusts whichever source is less uncertain, and the uncertainty shrinks after every measurement.

12.3 MASooda's three-state EKF

The state is $\mathbf{x} = (\text{SoC},\ V_{RC},\ 1/Q)$. Estimating $1/Q$ lets the BMS learn a sudden capacity loss from the voltage instead of being told. With measured current $\tilde{I}$:

$$\begin{aligned}
\textbf{predict:}\quad & \widehat{\text{SoC}}^- = \widehat{\text{SoC}} - \tilde{I}\,\Delta t\,\hat{q}/3600, \qquad \hat{V}_{RC}^- = \hat{V}_{RC} e^{-\Delta t/\tau} + R_1(1 - e^{-\Delta t/\tau})\tilde{I}, \\
& F = \begin{pmatrix} 1 & 0 & -\tilde{I}\Delta t/3600 \\ 0 & e^{-\Delta t/\tau} & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad P^- = FPF^\top + Q_{\text{proc}}, \\
\textbf{update:}\quad & \hat{V} = n_s\,\text{OCV}(\widehat{\text{SoC}}^- + \bar{\delta}) - \hat{V}_{RC}^- - R_0\tilde{I}, \qquad y = V_{\text{meas}} - \hat{V}, \\
& H = \big(n_s\,\text{OCV}'(\cdot),\ -1,\ 0\big), \quad S = HP^-H^\top + \sigma_V^2, \quad K = P^-H^\top/S, \\
& \hat{\mathbf{x}} = \hat{\mathbf{x}}^- + K\,y, \qquad P = P^- - K\,HP^-.
\end{aligned}$$

- F is the **Jacobian** of the prediction: how each new state depends on each old one. Its top-right entry links capacity errors to charge errors.
- H is the Jacobian of the voltage model. The OCV slope OCV′ is computed by a central difference over ±0.5% SoC. Where the OCV curve is flat, voltage tells little about SoC and the gain is small.
- P is kept symmetric after every update: $P \leftarrow \frac{1}{2}(P + P^\top)$.
- Sensor noise comes from a hash of (seed, drone, sample count), so a drone's measurements do not depend on which other drones fly, and replays are exact.



All drones are filtered at once: `np.einsum` handles a stack of 3 × 3 matrices, one per drone.

In code `drone_health.py › HealthModel._ekf` `HealthModel._gauss (Box–Muller from a hash)` `soc_bms` `soc_sigma`

In the paper's validation (40 drones × 3 seeds), the SoC error stayed within ±3σ in every sample, with RMSE 0.21–0.33%.

13 Energy decisions under uncertainty

Every "can I make it?" question uses a cautious bound on the BMS estimate, never the estimate itself.

13.1 The chance-constrained energy bound

$$E^{\mathrm{BMS}} = E^{\mathrm{use}}\left(\widehat{\mathrm{SoC}} - z\,\sigma_{\mathrm{SoC}},\ \frac{1}{\hat{q} + z\,\sigma_q}\right), \qquad z = 1.645.$$

Both the charge and the capacity are taken at their pessimistic one-sided bounds. For a Gaussian error, each bound is too optimistic only about 5% of the time.

> **EXAMPLE**
> $\widehat{\mathrm{SoC}} = 40\%$ with σ = 1%: decisions use $40 - 1.645 = 38.4\%$. The 1.6-point haircut is the price of not overestimating. In the paper's low-charge fleet it kept 6.4 more drones on the ground per run instead of letting them take off and divert.

13.2 Energy needed for the rest of the route

$$E^{\mathrm{need}}(L) = P_{\mathrm{el}}(v_c)\,\frac{L}{v_c}\,(1+\mu) + P_{\mathrm{el}}(0)\,t_{\mathrm{land}}, \qquad \mu = 0.2,\ t_{\mathrm{land}} = 30\ \mathrm{s}.$$

> **EXAMPLE**
> 1 km left at 10.8 m/s: flight time 92.6 s. Cruise energy $1060 \times 92.6/3600 = 27.3$ Wh, ×1.2 = 32.7 Wh, plus landing hover $1379 \times 30/3600 = 11.5$ Wh: $E^{\mathrm{need}} = 44.2$ Wh.

13.3 The feasibility test

A route is feasible when $E^{\mathrm{need}} \le E^{\mathrm{BMS}}$ **and** the weakest cell, at the charge left on arrival, stays above $3.0 + 0.05 = 3.05$ V while carrying the landing hover (before take-off it must also carry a climb). The cell voltage is $\mathrm{OCV}(s_k) - I\,r_{\mathrm{cell}}(1+\varepsilon_k)$ with I from the quadratic in 11.1. The voltage test catches a high-resistance cell that would hit cut-off under load long before its charge runs out.

SEEN ON THE LIVE SITE (PRESET 8)

"D-101 low reserve (57 Wh left, 66 Wh needed to reach V9): DIVERTING to V4 (265 m, needs 20 of 57 Wh)." That is this test failing for the original destination and passing for the nearest pad.

In code `drone_health.py › energy_needed_wh` `feasible` `sag_ok` `_pack_at` `lift_ok` `route_engine.py › _energy_check` `_divert`

13.4 Pad arbitration by endurance

When several urgent drones want the same landing pad, it goes to the one that can hover the shortest time:

$$\tau_i = \frac{3600\,E_i^{\mathrm{BMS}}}{P_{\mathrm{el},i}(0)}\ \mathrm{s}, \qquad \tau_i \leftarrow \min(\tau_i, 120\ \mathrm{s}) \text{ at CRITICAL or FAULT.}$$

The current holder keeps the pad unless a challenger has at least 15 s less endurance, so reservations do not flip-flop. Other arrivals stop 60 m short and loiter at the tube edge.

EXAMPLE

57 Wh left with 1379 W hover: $\tau = 3600 \times 57/1379 = 149$ s. A drone at BMS CRITICAL is capped at 120 s, so it wins the pad over this one by 29 s, more than the 15 s swap margin.

In code `route_engine.py › _endurance_s` `_reserve_pads` `_reservation_active` `_rank (who yields in a tactical shift)`

14 One tick, end to end

Every 0.1 s, the pieces above run in this order inside `simulation/route_engine.py › _simulate`.

#	Step	Maths	Chapter
1	Departures: pad clear and junction merge gap open?	distances, merge factor	3, 5
2	Neighbour search within 40 m	radius graph, spatial grid	1.5, 4
3	Spoofed broadcasts; trusted-range check and W-MSR	norms, trimmed mean	7
4	Local velocity: pursuit, car-following, boids, CPA	dot products, headway law, τ^*	3.5, 5
5	Conflict groups solve consensus ADMM	projections, weighted averages	4.4, 6
6	Dynamics, tube and geofence projection, crash check	Euler step, clamp, projection	2, 3.4
7	Battery, temperatures, EKF, BMS (every 0.5–1 s)	quadratic, exponentials, Kalman	10–12
8	Feasibility, diversions, pad arbitration, health weights (every 1 s)	chance bound, endurance	6.7, 13
9	Layer agreement (1 s) and DTC allocation (2 s)	prices, argmin, occupancy	8
10	STL robustness, KPIs, network metrics; stream a frame	min over pairs, Laplacian	4.2, 9

15 Function index

Every concept above, and where it lives in the repository.

Concept	File	Function / class
Vector clamp	simulation/dynamics.py	`clamp_vector_2d`
Double-integrator step	simulation/dynamics.py	`step_dynamics`
Radius / kNN graph	simulation/comm_graph.py	`build_radius_graph`, `build_knn_graph`
Spatial grid index	simulation/spatial_grid.py	`UniformGrid`, `radius_adjacency`, `close_same_layer_pairs`, `layered_min_separation_grid`
Graph metrics, Laplacian λ_2	simulation/route_engine.py	`_graph_metrics`
Map projection	simulation/road_network.py	`GeoFrame.to_local`, `to_geo`
Dijkstra	simulation/road_network.py	`RoadNetwork.shortest_path`
Arc-length polyline, projection	simulation/road_network.py	`RoutePath.point_at`, `tangent_at`, `project`
Sector index	simulation/route_engine.py	`_sector`
Polygons, restricted areas	simulation/dtc_console.py	`points_in_polygon`, `segments_cross_polygon`, `polygon_area`, `plan_route`
Boids terms	local_behavior/boids.py	`separation`, `alignment`, `cohesion`, `corridor_following`
Car-following, CPA, tube	simulation/route_engine.py	`_simulate`, `_layer_transition_clear`
Junction merges, platoons	simulation/route_formation.py	`MergeTracker.factor`, `platoon_stats`
Conflict detection	conflict_resolution/detection.py	`detect_conflicts`
Half-space constraints, projections	conflict_resolution/constraints.py	`build_pair_constraints`, `project_pair`, `project_ball`
Consensus ADMM	conflict_resolution/solver.py	`ADMMSolver.solve`
ADMM in the engine	conflict_resolution/adapter.py	`resolve_conflict_event`, `resolve_all_conflicts`

Averaging consensus	resilience/consensus.py	`plain_average`, `run_consensus`, `run_wmsr_consensus`
W-MSR	resilience/wmsr.py	`wmsr_scalar`, `wmsr_vector`
Attack injection	resilience/attack.py	`inject_attack`, `apply_attack`
Trusted-range defence	resilience/defense.py	`defend_neighbor_reports`
Priority-aware allocation	simulation/overseer.py	`GlobalOverseer.update`
Link outage	simulation/overseer_link.py	`OverseerLink.is_up`, `request`
Layer agreement, prices	simulation/layer_agreement.py	`LayerAgreement.round`, `dtc_seed_prices`, `follow_penalty`, `climb_cost`, `departure_layer`
STL robustness	verification/stl_monitor.py	`instantaneous_robustness`, `robustness_series`, `stl_monitor`, `liveness_robustness`
Layer-aware ρ, recovery	verification/metrics.py	`robustness_series`, `recovery_times`, `compute_run_metrics`
Air density	simulation/drone_health.py	`air_density`
Flight power	simulation/drone_health.py	`HealthModel.power`, `min_power_speed`, `climb_power_w`
1-RC battery, heat	simulation/drone_health.py	`HealthModel._discharge`, `_r`, `_ocv_cell`, `_pack_ocv`
Usable energy	simulation/drone_health.py	`_usable_wh`, `e_rem_wh`, `e_rem_bms_wh`
EKF	simulation/drone_health.py	`HealthModel._ekf`, `_gauss`
BMS levels	simulation/drone_health.py	`HealthModel._level`, `bms`
Feasibility	simulation/drone_health.py	`energy_needed_wh`, `feasible`, `sag_ok`, `lift_ok`
Pad arbitration, health weights	simulation/route_engine.py	`_endurance_s`, `_reserve_pads`, `_health_tick`

Further reading: S. Boyd et al., "Distributed optimization and statistical learning via ADMM" (2011); H. LeBlanc et al., "Resilient asymptotic consensus in robust networks" (2013); A. Donzé and O. Maler, "Robust satisfaction of temporal logic over real-valued signals" (2010); G. Plett, *Battery Management Systems, Vol. I* (2015); C. Reynolds, "Flocks, herds and schools" (1987); J. G. Leishman, *Principles of Helicopter Aerodynamics* (2006).